AI PROJECT · SPRING 2026

# Draw & Solve

## Handwritten Math Expression Recognition


**Matti H. V. Lehmann**
Introduction · Conclusion

**David A. Spickenheuer**
System Pipeline

**Jonas Schenke**
Image Processing

**Philip Nguyen**
Dataset & Data Prep

**Staniya Thomas**
CNN & Evaluation

**Abhirup Sain**
Live Demo

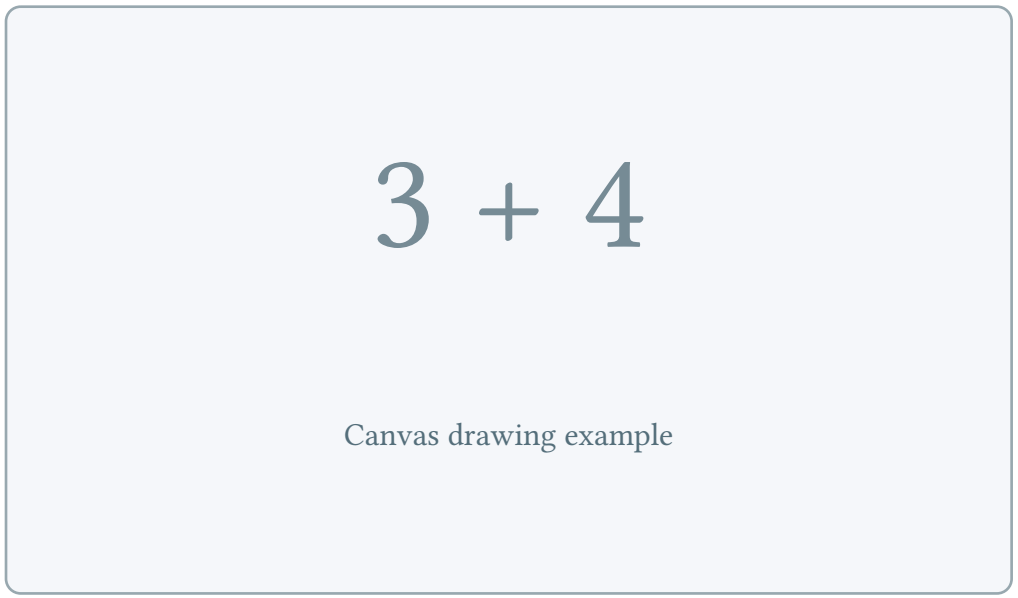
# Team & Task Division

| Name | Student ID | Task |
|---|---|---|
| Matti Hannes Vincent Lehmann | T14H06309 | Introduction · Conclusion |
| David Axel Spickenheuer | T14H06329 | System Pipeline |
| Jonas Schenke | T14H06307 | Image Processing & Segmentation |
| Philip Nguyen | 91499153X | Dataset & Data Preparation |
| Staniya Thomas | T14H06310 | CNN Architecture & Evaluation |
| Abhirup Sain | T14H06318 | Live Demo |

# The Problem

- Handwriting varies widely per writer
- Operators share features with digits (× vs x, − vs ÷)
- Multi-stroke digits confuse naive segmenters

**Goal:** Draw a math expression on screen ⟶ get the result instantly.



# Our Solution: Draw & Solve

1. Draw an expression on a browser canvas
2. OpenCV segments individual symbols
3. CNN classifies each symbol (14 classes)
4. Safe AST evaluator computes the result

**Digits**

0 · 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9

**Operators**

+ · − · × · ÷

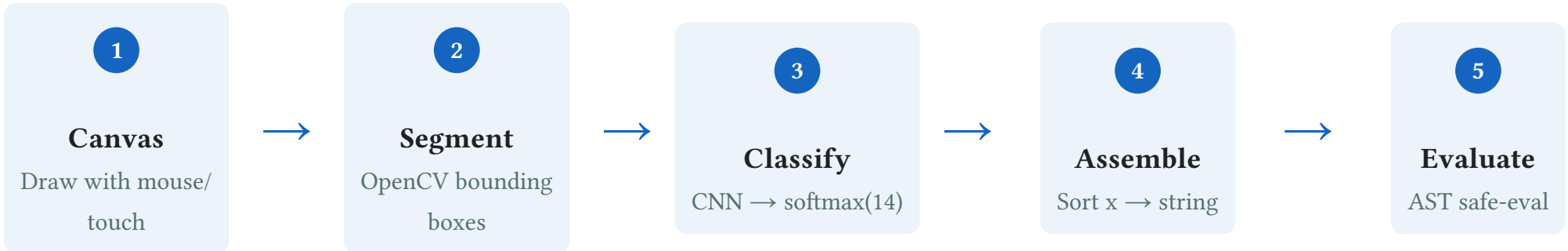
*App screenshot*

# v1 → v2: What Changed

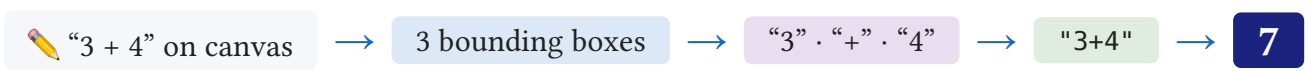
| Aspect | v1 | v2 |
|---|---|---|
| Classifier | MNIST CNN + geometry heuristics | **Unified 14-class CNN** |
| Dataset | MNIST only | Sagyamthapa (Kaggle) |
| Accuracy | 99% digits · operators fragile | **99.44%** all 14 classes |

**One model for everything** — no hand-crafted rules for operators, just learned features.

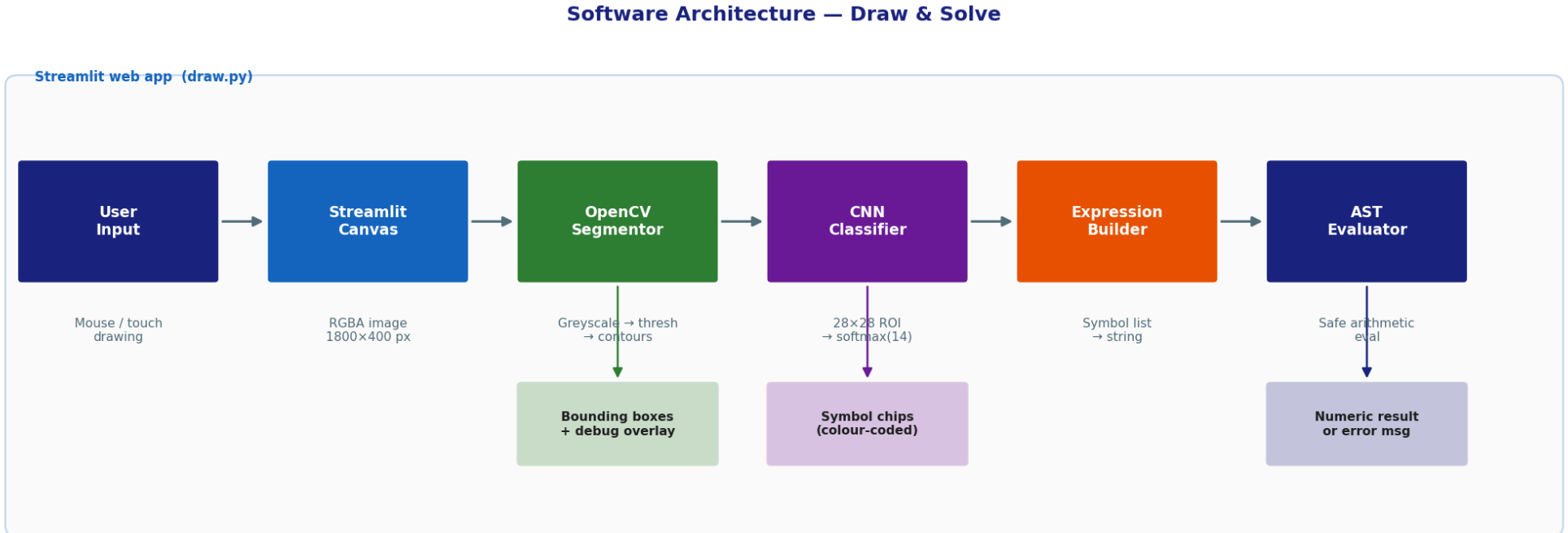
# System Pipeline




*Example — user draws “3 + 4”:*



# Software Architecture



# Preprocessing & Inference

### Per-symbol preprocessing

- Crop bounding box from greyscale image
- Add white padding, scale to fit 26 × 26
- Centre on 28 × 28 white canvas
- Divide ÷ 255 ⟶ ink ≈ 0, background ≈ 1

### Inference & assembly

- CNN softmax ⟶ 14 class probabilities
- Confidence < 0.60 ⟶ label as ?
- Sort symbols by x ⟶ expression string
- AST evaluator: only +, −, ×, ÷ permitted (no `eval()`)

*Raw ⟶ ROI ⟶ 28 × 28 model input*

# Thresholding & Contour Detection

### Binarisation

- Canvas: white (255), ink ≈ 0
- `THRESH_BINARY_INV`: ink ⟶ 255, bg ⟶ 0
- 3 × 3 dilation closes gaps between strokes
- `findContours` detects white regions on black

### Bounding boxes

- Outermost contours only (`RETR_EXTERNAL`)
- Box area < 30 px² discarded (noise filter)
- Sort all boxes left ⟶ right by x-coordinate

*Gray ⟶ threshold ⟶ dilated ⟶ boxes*

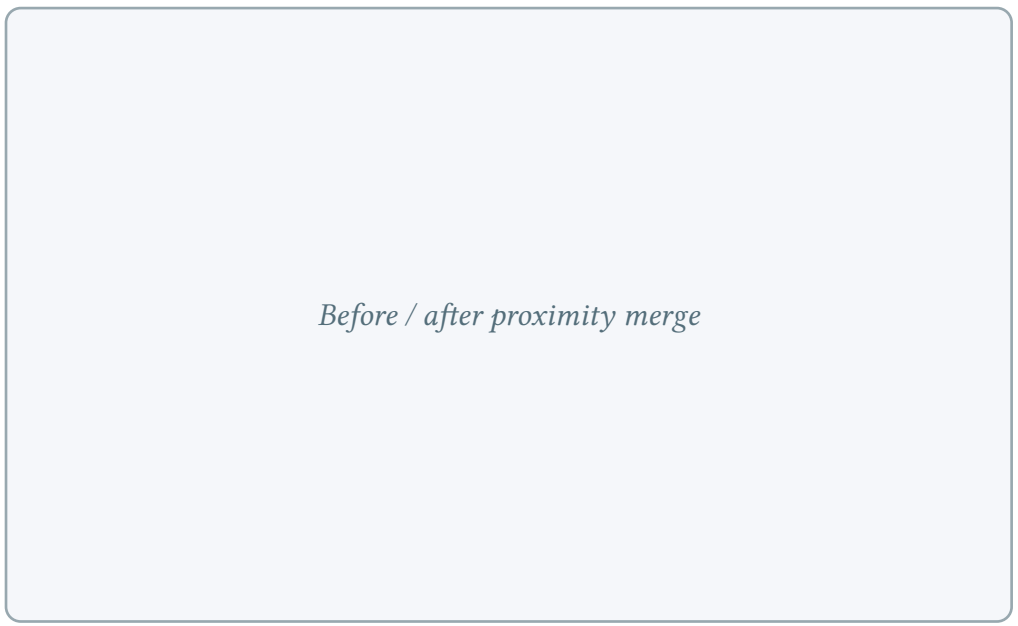
# Proximity Merging

**Problem:** digits 4, 5, 7 are multi-stroke → each stroke gets its own box

**Solution:** merge horizontally adjacent boxes with gap ≤ **20 px**

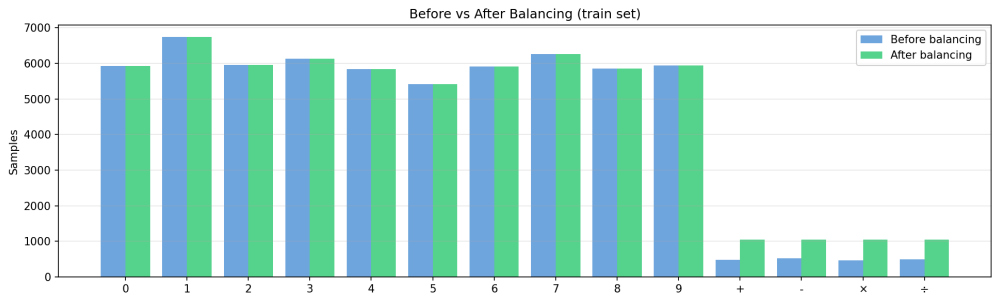
- Iterate boxes left → right
- If next box starts within 20 px, expand current box
- Otherwise, emit current box and start a new one

> Also handles the two dots in ÷ after dilation merges them with the bar.



# Dataset & Train/Test Split

| Source | Detail |
|---|---|
| Digits 0–9 | MNIST (LeCun et al.) |
| Operators +−×÷ | Sagyamthapa (Kaggle) |
| Classes used | **14** (digits + operators) |
| Balanced per class | **1 048 samples** |
| Total training pool | 14 672 images |
| Train / Val | 85 % / 15 % of pool |
| Test set (held-out) | **10 490 images** |
| — MNIST test | 10 000 images |
| — Kaggle ops (20 %) | 490 images (seed = 42) |



# Class Balancing & Augmentation

**Imbalance problem**

- Digits: 5 000 – 6 700 samples each
- Operators: 1 048 samples each ⟶ **6:1 ratio**
- Without fix: model predicts digits almost always

**Fix:** downsample digits to 1 048 per class

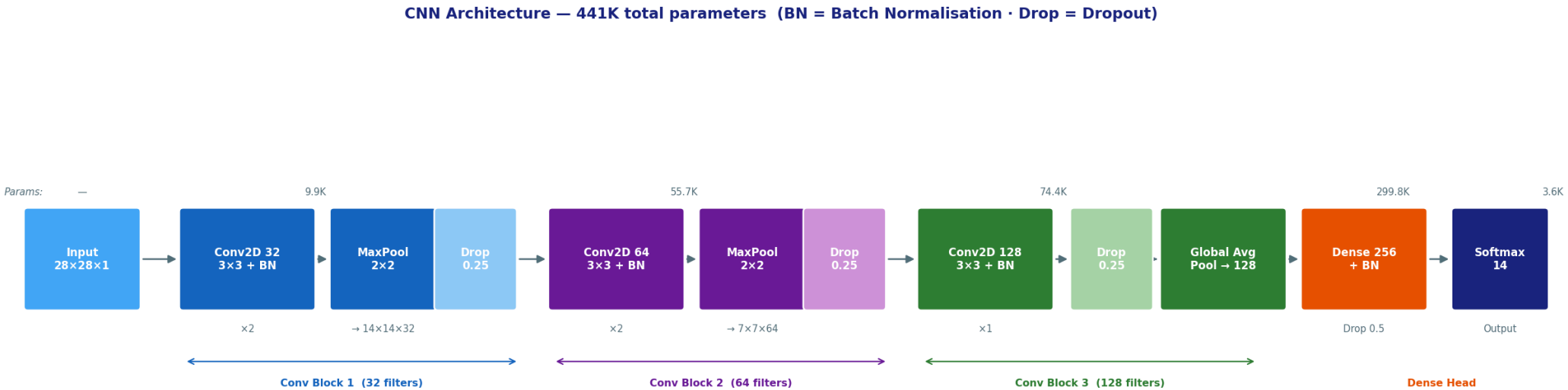
Result: **perfectly balanced** — 1 048 × 14 = 14 672 images

**Augmentation** (training only, fill = black)

| Transform | Range |
|---|---|
| Rotation | ± 8° |
| Zoom | ± 10% |
| Translation | ± 8% x/y |

- Simulates real writing variation
- Fill = **0 (black)** — matches MNIST convention
- Key to generalizing from only 1 048 samples

# CNN Architecture — Overview



# CNN Architecture — Details

**Input:** 28 × 28 × 1 greyscale

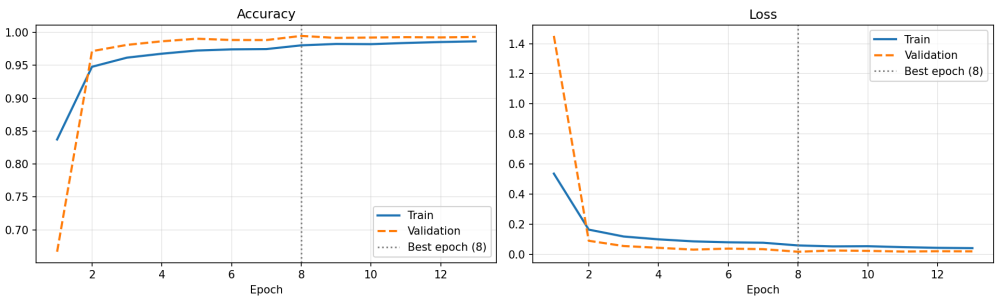
| Block | Output shape | Params |
|---|---|---|
| Conv32×2 + BN, MaxPool, Drop 0.25 | (14,14,32) | 9.9K |
| Conv64×2 + BN, MaxPool, Drop 0.25 | (7, 7, 64) | 55.7K |
| Conv128 + BN, GlobalAvgPool, Drop 0.25 | (128) | 74.4K |
| Dense 256 + BN, Drop 0.5 | (256) | 34.1K |
| Dense 14, softmax | (14) | 3.6K |
| **Total** | | **178K** |

**Design choices**

- BatchNorm after every conv ⟶ stable training
- Dropout 0.25 (conv) / 0.5 (dense) ⟶ regularization
- Filters double per block: 32 ⟶ 64 ⟶ 128
- **GlobalAveragePooling2D** in block 3 — 128 features instead of 1152 ⟶ less overfitting
- AdamW (lr = 0.001, weight decay = 1e-4)
- Early stopping: patience = 5

178K params — trains in **< 3 min**.

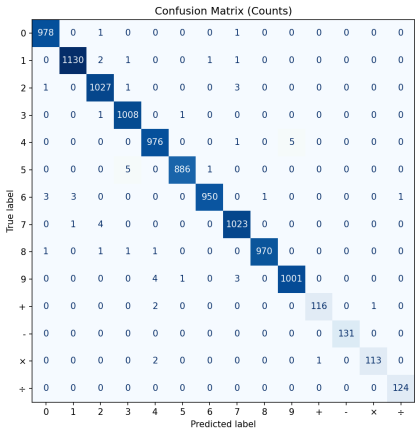
# Training Results

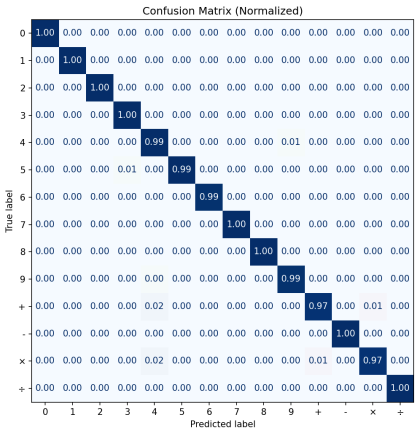



| Metric | Value |
|---|---|
| Total epochs | 13 (early stop patience = 5) |
| Best epoch | 8 |
| Best val acc | **99.46%** |
| Train/val gap | < 0.5% |

- Val acc reaches **98%** after just 1 epoch
- AdamW weight decay prevents overfitting
- No overfitting despite only 14 672 training samples

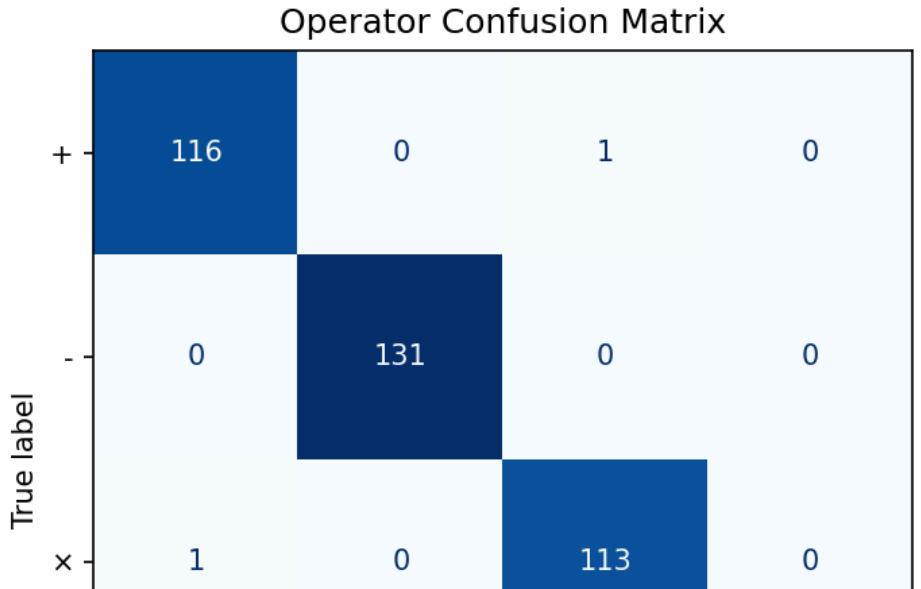
# Confusion Matrix



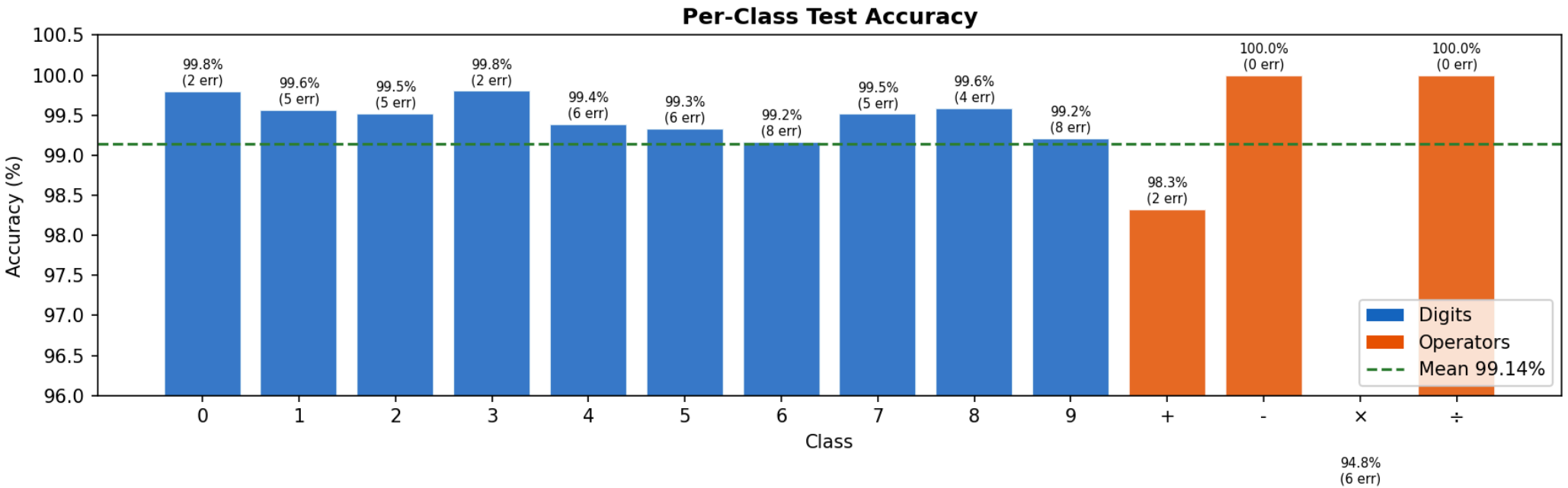




**Overall: 99.44%** (10 490 test images)
Digits: 99.49 %
Operators: 98.78 %

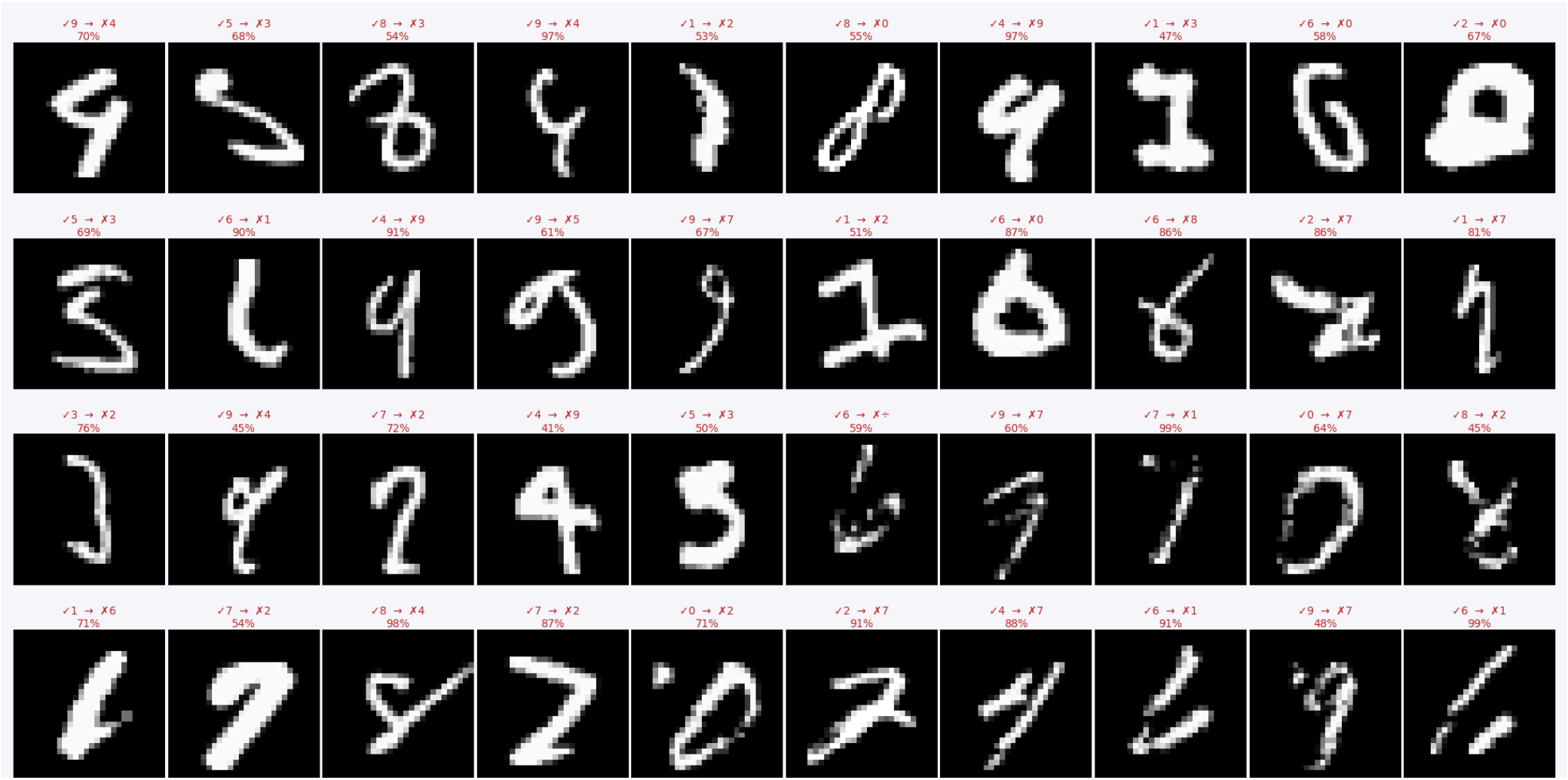
The matrix is nearly diagonal — 59 misclassified out of 10 490.



# Per-Class Accuracy



# Failure Cases — All 59 Misclassified Symbols



# Error Analysis

**Top confusion pairs** (test set, 59 total errors)

| True | Predicted | Count | Why? |
|---|---|---|---|
| 4 | 9 | 5 | Open loop vs. closed |
| 5 | 3 | 5 | Similar curvature |
| 9 | 4 | 4 | Open loop vs. stem |
| 7 | 2 | 4 | Diagonal bar confusion |
| × | 7 | 3 | Cross ≈ slanted stroke |

**Key observations**

- × is the weakest class: 94.8% accuracy (6 errors in 116)
- − and ÷: 100% accuracy on the test set
- Most errors are **digit-to-digit**, not cross-type
- Worst pairs are visually near-identical shapes

With only 1 048 operator samples, × is most sensitive to style variation — being confused as a slanted 7.

# Demo

1. Draw `3 + 4` → verify result **7**
2. Draw `12 × 3` → verify **36**
3. Draw `8 ÷ 2` → verify **4**
4. Draw an ambiguous symbol → show `?` chip
5. Point out per-symbol color coding

```
streamlit run draw.py
```

Model: `outputs/best_model.keras` · 14 classes

*App screenshot*

# Conclusion

### What we built

- End-to-end math solver running in the browser
- **99.44% val accuracy** across 14 classes
- One unified CNN — digits **and** operators
- Safe AST evaluator (no `exec` / `eval`)

> Adding a new symbol class = more training data only, no new code.

### Future work

- Multi-digit numbers as a single unit
- Fractions and parentheses
- More classes: =, decimal point, variables
- Larger operator dataset ⟶ reduce failure cases
- Mobile / touch input optimization

# Thank You

Questions?

Matti H. V. Lehmann

David A. Spickenheuer

Jonas Schenke

Philip Nguyen

Staniya Thomas

Abhirup Sain